

Universitatea Politehnica București

Facultatea de Automatică și Calculatoare

Implementarea unui Dispozitiv portabil de comunicații bidirecționale

Proiectare Logică - Proiect 2026

Proiect realizat de:

Sandu Eric-Andrei

Mateciuc Vlad-Ștefan

Hâncu David-Andrei

Cuprins

1	Descrierea funcționării automatului	2
2	Schema bloc a automatului	2
2.1	Descrierea Semnalelor	3
2.2	Interacțiunea în Modul Pairing	3
3	Organigrama automatului	4
3.1	Descrierea detaliată a stărilor automatului	5
4	Identificarea ecuațiilor de stare	6
4.1	Codificarea Stărilor	6
4.2	Definirea Variabilelor de Intrare	6
4.3	Hărțile Karnaugh pentru Funcția de Stare Următoare	6
4.3.1	Ecuția pentru Q_3	7
4.3.2	Ecuția pentru Q_2	7
4.3.3	Ecuția pentru Q_1	8
4.3.4	Ecuția pentru Q_0	8
5	Implementarea cu CBB JK și Multiplexoare 4:1	9
5.1	Implementarea etajului Q_3	9
5.2	Implementarea etajului Q_2	10
5.3	Implementarea etajului Q_1	11
5.4	Implementarea etajului Q_0	12
6	Implementarea ieșirilor cu porți logice	13
6.1	Implementarea ieșirii de eroare	13
6.2	Implementarea ieșirilor de recepție	13
6.3	Implementarea ieșirilor de transmisie	14
7	Implementarea cu microinstrucțiuni variabile	15
7.1	Organigrama pentru implementarea cu microinstrucțiuni	15
7.2	Calculul lungimii microinstrucțiunii	15
7.2.1	Date de intrare	15
7.2.2	Dimensionarea câmpurilor	16
7.2.3	Aplicarea calculului final	16
7.3	Proiectarea unității de comandă microprogramată	17
7.4	Alegerea componentelor digitale	17
7.4.1	Memorie	17
7.4.2	Multiplexare	17
7.4.3	Numărătoare	18
7.4.4	Masca și Porțile Logice	18
7.4.5	Detalii Tehnice și Pinout	18
7.5	Completarea memoriei de microprogram	19

1 Descrierea funcționării automatului

Să se proiecteze un automat secvențial sincron care controlează funcționarea unui aparat portabil de emisie-recepție (Walkie-Talkie).

Scopul automatului este de a facilita comunicarea bidirecțională între două dispozitive, permițând utilizatorului să transmită și să primească un flux audio.

Sistemul pornește într-o stare de așteptare (Idle/Listening), monitorizând semnalele de intrare. Comportamentul dispozitivului este dictat prin următoarele stări:

- **Modul Normal (Receiving/Transmitting):** În starea implicită, dispozitivul primește un flux audio de la canalul său curent. Dacă se apasă butonul PTT (Push-to-Talk), automatul trece în modul de transmisie, activând microfonul și LED-ul de transmisie. La eliberarea butonului, revine în modul de recepție.
- **Modul Pairing:** La apăsarea butonului PAIR, dispozitivul intră într-o secvență de scanare, așteptând un răspuns (handshake), urmată de o stare de succes sau eroare.
- **Selectarea Canalului:** Dispozitivul permite comutarea între două canale salvate. Canalul activ este schimbat prin apăsarea CH_BTN.

2 Schema bloc a automatului

Figura 1 prezintă intrările și ieșirile unității centrale de control. Automatul primește semnale de la interfața cu utilizatorul (butoane) și de la modulul radio (semnale de stare), generând comenzi pentru componentele hardware (LED-uri, Audio).

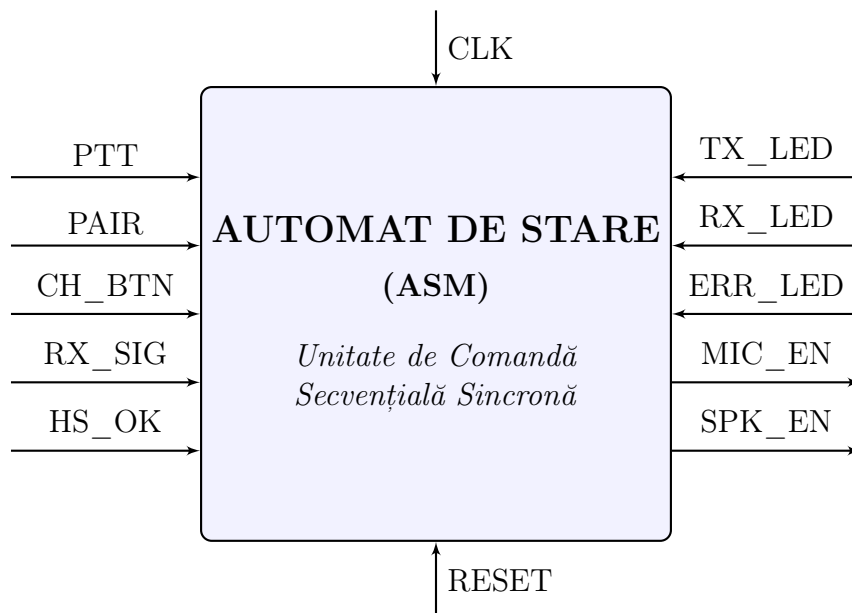


Figura 1: Schema bloc detaliată a automatului de stare pentru un singur dispozitiv.

2.1 Descrierea Semnalelor

Tabelul de mai jos detaliază rolul fiecărui semnal identificat în schema bloc:

Intrări (Senzori și Interfață):

- **PTT (Push-To-Talk):** Semnal activ pe '1' când utilizatorul ține apăsat butonul de vorbire. Comandă tranziția din starea de ascultare în cea de emisie.
- **PAIR:** Buton monostabil pentru inițierea procedurii de împerechere cu un alt dispozitiv.
- **CH_BTN:** Buton pentru comutarea ciclică a canalului de comunicație (Canal 1 / Canal 2).
- **RX_SIG:** Semnal digital venit de la modulul receptor radio care indică prezența unei purtătoare (carrier detect) sau a unui semnal audio valid pe canalul curent.
- **HS_OK (Handshake OK):** Semnal logic generat de subsistemul de decodificare a datelor. Devine '1' doar dacă, în timpul modului de Pairing, se primește un cod corect de confirmare de la celălalt dispozitiv.

Ieșiri (Comenzi Hardware):

- **TX_LED:** Activează LED-ul roșu pentru a indica transmisia în curs.
- **RX_LED:** Activează LED-ul verde pentru a indica recepția unui semnal sau un canal liber.
- **MIC_EN:** Activează preamplificatorul de microfon și modulatorul emițătorului.
- **SPK_EN:** Activează amplificatorul audio final pentru difuzor (Speaker).
- **ERR_LED:** LED galben/intermitent care semnalizează eșecul procesului de pairing (timeout sau lipsă handshake).

2.2 Interacțiunea în Modul Pairing

Pentru a justifica complexitatea automatului și necesitatea stării de "Handshake" (HS_OK), Figura 2 ilustrează interacțiunea conceptuală între două unități.

Generare HS_OK intern

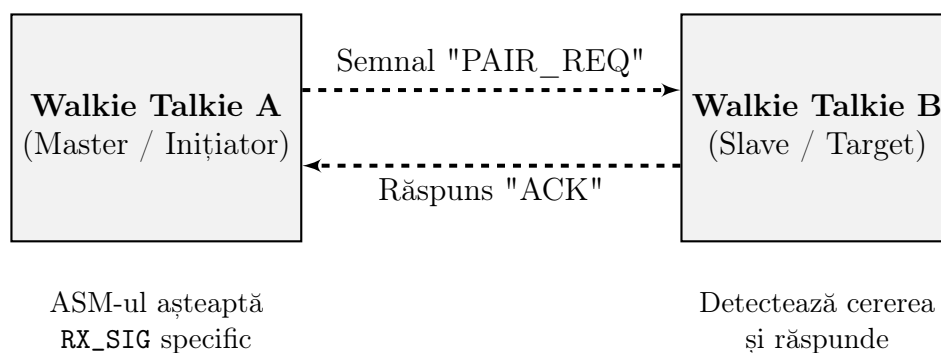


Figura 2: Schema logică a procesului de Pairing. Semnalul HS_OK este generat intern de Dispozitivul A doar la recepția confirmării (ACK) de la Dispozitivul B.

3 Organigrama automatului

Organigrama din Figura 3 descrie fluxul logic al automatului, evidențiind cele 5 ieșiri principale generate în stările cheie.

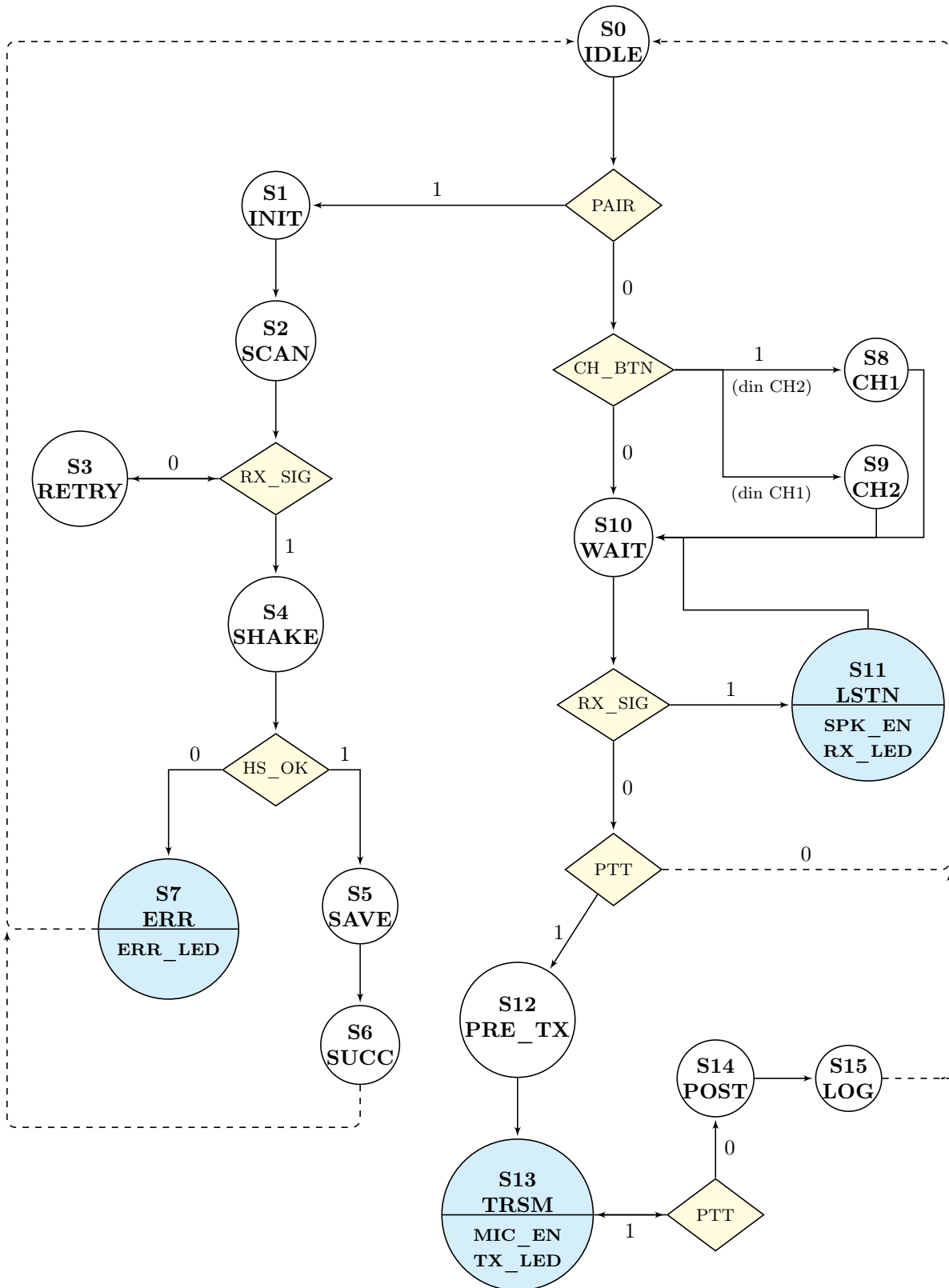


Figura 3: Organigrama de tip Moore a dispozitivului de comunicații

3.1 Descrierea detaliată a stărilor automatului

S0 (IDLE)

Este starea de repaus și punctul central de decizie. Aici sistemul monitorizează intrările (PAIR, CH_BTN, RX_SIG, PTT) pentru a determina modul de operare.

Ramura de Pairing (S1 - S7)

Această secvență este inițiată la apăsarea butonului PAIR:

- **S1 (INIT):** Inițializează variabilele necesare protocolului de împerechere.
- **S2 (SCAN) și S3 (RETRY):** Dispozitivul scanează frecvențele căutând un semnal; dacă nu este găsit, trece prin **RETRY** care scanează cu o putere de transmitere crescătoare până găsește un dispozitiv.
- **S4 (SHAKE):** Odată detectat un semnal, se așteaptă confirmarea handshake-ului (HS_OK).
- **S7 (ERR):** Stare de eroare. Se activează dacă handshake-ul eșuează, generând semnalul ERR_LED.
- **S5 (SAVE) și S6 (SUCC):** Dacă handshake-ul este valid, datele sunt salvate, iar împerecherea este marcată ca reușită.

Ramura de Selectare Canal (S8 - S9)

Accesată prin apăsarea CH_BTN pentru a comuta frecvența activă:

- **S8 (CH1) și S9 (CH2):** Reprezintă cele două canale disponibile. Trecerea prin aceste stări actualizează canalul activ.

Monitorizare și Recepție (S10 - S11)

Gestionează traficul audio de intrare:

- **S10 (WAIT):** Stare intermediară care verifică prezența semnalului radio sau comanda de transmisie.
- **S11 (LSTN):** Stare activă de recepție. Când RX_SIG este detectat, automatul activează ieșirile SPK_EN și RX_LED.

Ramura de Transmisie (S12 - S15)

Activată prin menținerea apăsată a butonului PTT:

- **S12 (PRE_TX):** Pregătește circuitele de emisie.
- **S13 (TRSM):** Stare activă de transmisie. Menține active ieșirile MIC_EN și TX_LED atâta timp cât PTT este apăsat.
- **S14 (POST) și S15 (LOG):** După eliberarea butonului, se execută operațiuni post-transmisie și logarea evenimentului înainte de revenirea în IDLE.

4 Identificarea ecuațiilor de stare

Pentru a transpune organigrama într-un circuit digital secvențial, fiecărei stări i s-a alocat un cod binar unic pe 4 biți: $Q_3Q_2Q_1Q_0$.

4.1 Codificarea Stărilor

Stare	Simbol	Cod ($Q_3Q_2Q_1Q_0$)
S0	IDLE	0000
S1	INIT	0001
S2	SCAN	0010
S3	RETRY	0011
S4	SHAKE	0100
S5	SAVE	0101
S6	SUCC	0110
S7	ERR	0111
S8	CH1	1000
S9	CH2	1001
S10	WAIT	1010
S11	LSTN	1011
S12	PRE_TX	1100
S13	TRSM	1101
S14	POST	1110
S15	LOG	1111

Tabela 1: Tabela de alocare a stărilor

4.2 Definirea Variabilelor de Intrare

Ecuațiile de tranziție depind de starea curentă și de intrările asincrone. Pentru simplificarea tabelelor, vom folosi următoarele notații:

- P = Buton PAIR
- C = Buton CH_BTN
- R = Semnal RX_SIG
- H = Semnal HS_OK
- T = Buton PTT

4.3 Hărțile Karnaugh pentru Funcția de Stare Următoare

Vom determina funcțiile logice $Q_3^+, Q_2^+, Q_1^+, Q_0^+$ completând hărțile Karnaugh.

4.3.1 Ecuția pentru Q_3

$Q_1Q_0 \backslash Q_3Q_2$	00	01	11	10
00	$\bar{P}$	0	1	1
01	0	0	1	1
11	0	0	0	1
10	0	0	1	$R + T$

Tabela 2: Harta K pentru Q_3^+

Ecuția minimizată extrasă din hartă:

$$\begin{aligned}
 Q_3^+ = & Q_3Q_1 + Q_3Q_2\overline{Q_0} + Q_3\overline{Q_2}Q_0 + \\
 & \overline{Q_2}Q_1\overline{Q_0} \cdot \bar{P} + \\
 & Q_3\overline{Q_2} \cdot (R + T)
 \end{aligned} \tag{1}$$

4.3.2 Ecuția pentru Q_2

$Q_1Q_0 \backslash Q_3Q_2$	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	R	0	0	0
10	R	0	1	$\bar{R}T$

Tabela 3: Harta K pentru Q_2^+

Ecuția minimizată extrasă din hartă:

$$\begin{aligned}
 Q_2^+ = & Q_2\overline{Q_1} + Q_3Q_2\overline{Q_0} + \\
 & \overline{Q_3}Q_2\overline{Q_1} \cdot R + \\
 & Q_3Q_1\overline{Q_0} \cdot \bar{R}T
 \end{aligned} \tag{2}$$

4.3.3 Ecuția pentru Q_1

$Q_1Q_0 \backslash Q_3Q_2$	00	01	11	10
00	$\bar{P}\bar{C}$	$\bar{H}$	0	1
01	1	1	$\bar{T}$	1
11	$\bar{R}$	0	0	1
10	$\bar{R}$	0	1	R

Tabela 4: Harta K pentru Q_1^+

Ecuția minimizată extrasă din hartă:

$$\begin{aligned}
 Q_1^+ = & \overline{Q_3Q_1}Q_0 + Q_3\overline{Q_2Q_1} + Q_3\overline{Q_2}Q_0 + Q_3Q_2Q_1\overline{Q_0} + \\
 & Q_3\overline{Q_2} \cdot R + \\
 & \overline{Q_3Q_2}Q_1 \cdot \bar{R} + \\
 & \overline{Q_1}Q_0 \cdot \bar{T} + \\
 & \overline{Q_2Q_1} \cdot \bar{P}\bar{C}
 \end{aligned} \tag{3}$$

4.3.4 Ecuția pentru Q_0

$Q_1Q_0 \backslash Q_3Q_2$	00	01	11	10
00	P	1	1	0
01	0	0	T	0
11	$\bar{R}$	0	0	0
10	$\bar{R}$	0	1	R

Tabela 5: Harta K pentru Q_0^+

Ecuția minimizată extrasă din hartă:

$$\begin{aligned}
 Q_0^+ = & Q_2\overline{Q_1}Q_0 + Q_3Q_2\overline{Q_0} + \\
 & Q_3\overline{Q_2}Q_1 \cdot R + \\
 & \overline{Q_3Q_2}Q_1 \cdot \bar{R} + \\
 & Q_3Q_2\overline{Q_1} \cdot T + \\
 & \overline{Q_3Q_1}Q_0 \cdot P
 \end{aligned} \tag{4}$$

5 Implementarea cu CBB JK și Multiplexoare 4:1

Selectorii: Q_1, Q_0

5.1 Implementarea etajului Q_3

Logica intrărilor MUX depinde de Q_2 .

Zona J ($Q_3 = 0$)

$Q_3 Q_2$ $Q_1 Q_0$	00	01
00 (D_0)	$\bar{P}$	0
01 (D_1)	0	0
11 (D_3)	0	0
10 (D_2)	0	0

Zona K ($Q_3 = 1$)

$Q_3 Q_2$ $Q_1 Q_0$	11	10
00 (D_0)	0	0
01 (D_1)	0	0
11 (D_3)	1	0
10 (D_2)	0	$\bar{R}\bar{T}$

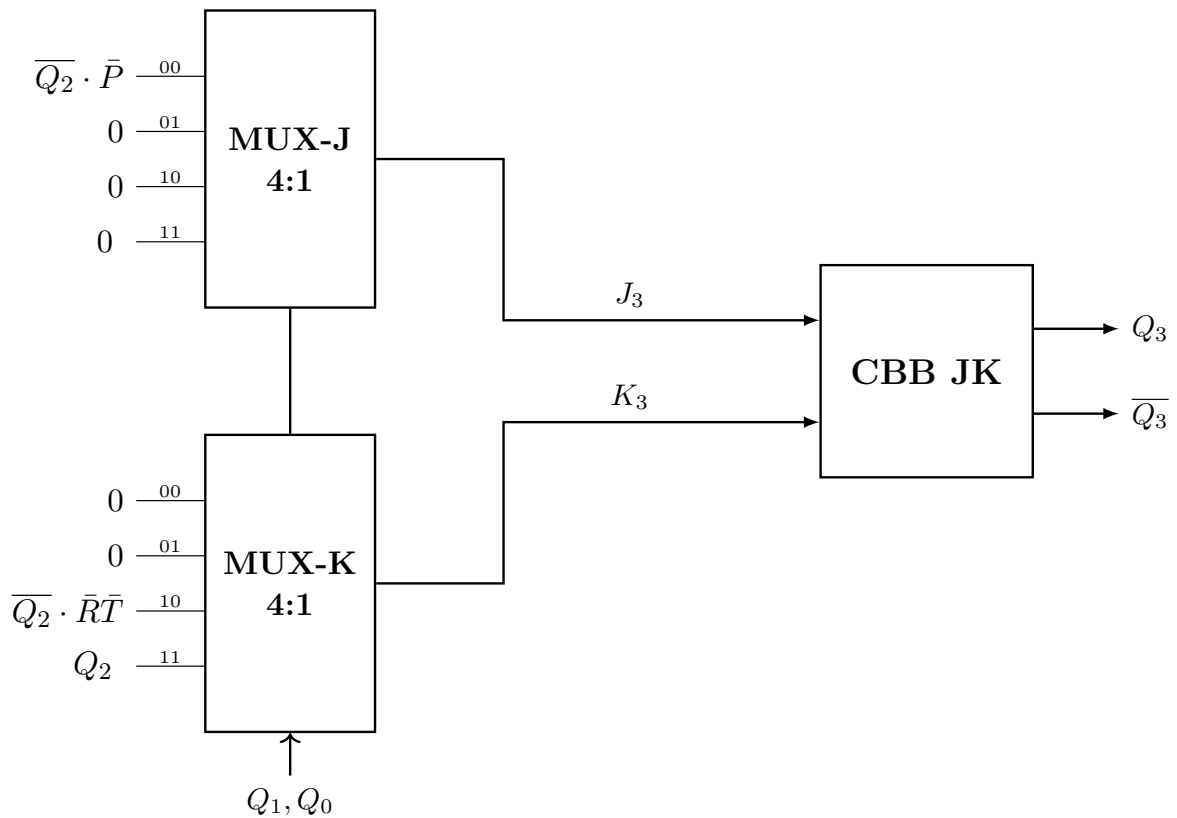


Figura 4: Implementarea grafică a etajului Q_3

5.2 Implementarea etajului Q_2

Logica intrărilor **MUX** depinde de Q_3 .

Zona J ($Q_2 = 0$)

Q_3Q_2 \ Q_1Q_0	00	10
00 (D_0)	0	0
01 (D_1)	0	0
11 (D_3)	R	0
10 (D_2)	R	$\bar{R}T$

Zona K ($Q_2 = 1$)

Q_3Q_2 \ Q_1Q_0	01	11
00 (D_0)	0	0
01 (D_1)	0	0
11 (D_3)	1	1
10 (D_2)	1	0

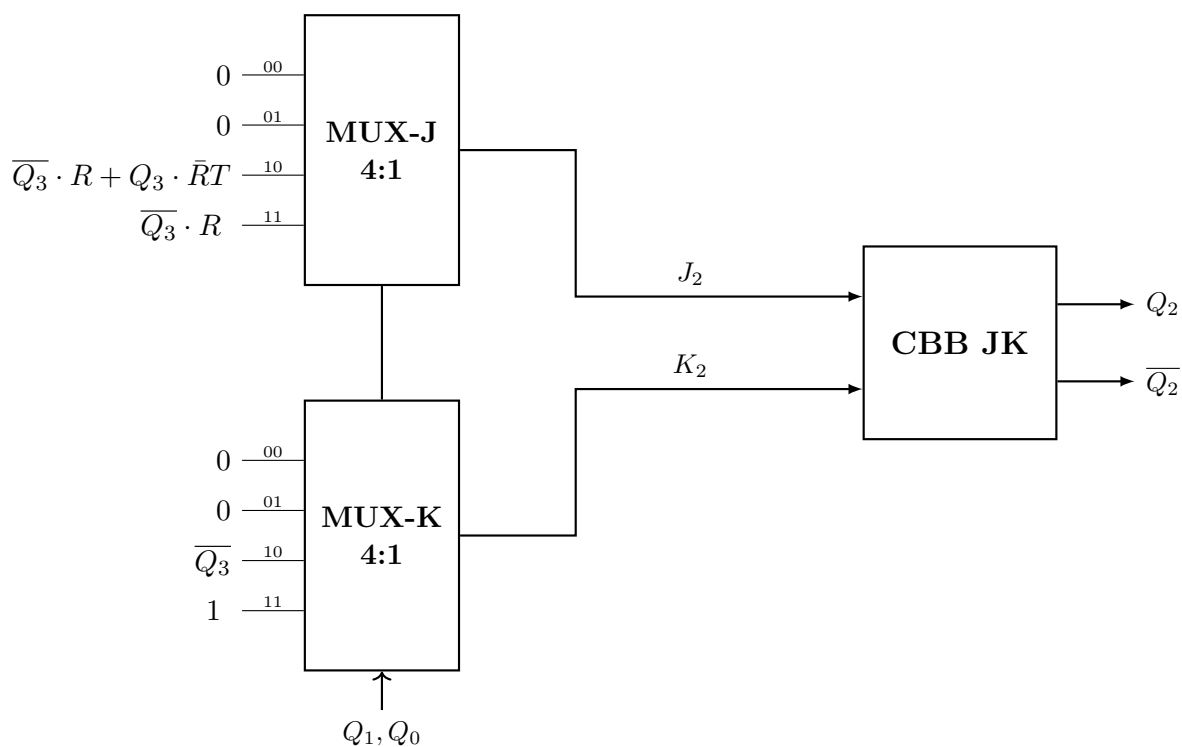


Figura 5: Implementarea grafică a etajului Q_2

5.3 Implementarea etajului Q_1

Logica intrărilor **MUX** depinde de Q_3 și Q_2 .

Zona J ($Q_1 = 0$)

$Q_3Q_2 \backslash Q_1Q_0$	00	01	11	10
00	$\bar{P}\bar{C}$	$\bar{H}$	0	1
01	1	1	$\bar{T}$	1

Zona K ($Q_1 = 1$)

$Q_3Q_2 \backslash Q_1Q_0$	00	01	11	10
11	R	1	1	0
10	R	1	0	$\bar{R}$

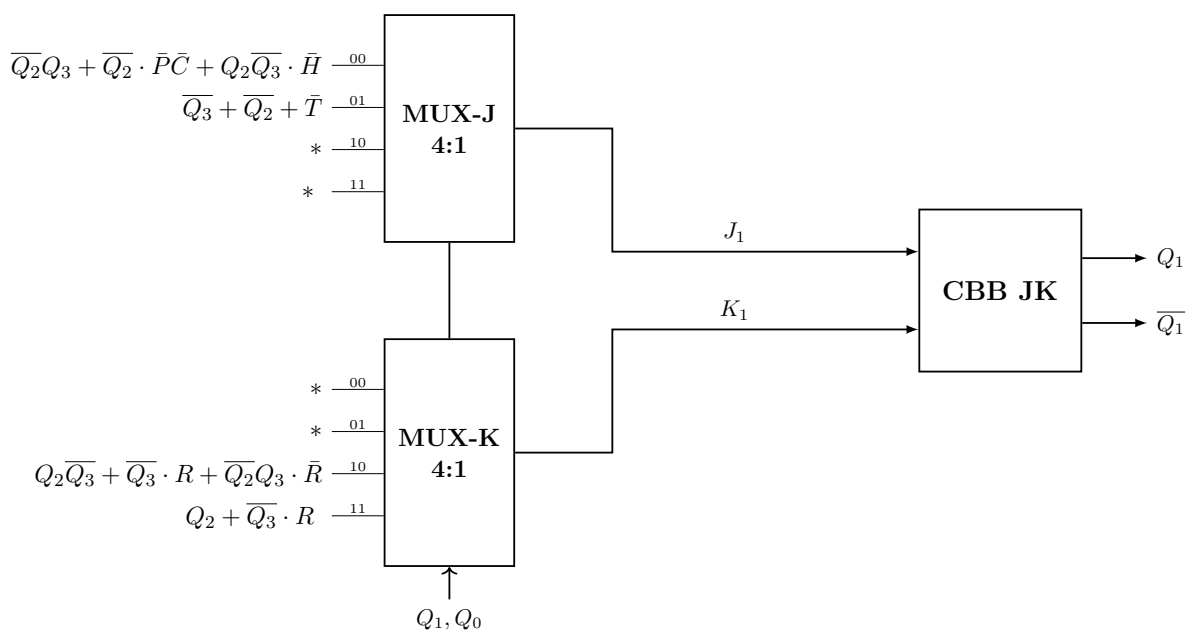


Figura 6: Implementarea grafică a etajului Q_1

5.4 Implementarea etajului Q_0

Logica intrărilor MUX depinde de Q_3 și Q_2 .

Zona J ($Q_0 = 0$)

$Q_3Q_2 \backslash Q_1Q_0$	00	01	11	10
00	P	1	1	0
10	$\bar{R}$	0	1	R

Zona K ($Q_0 = 1$)

$Q_3Q_2 \backslash Q_1Q_0$	00	01	11	10
01	1	1	$\bar{T}$	1
11	R	1	1	1

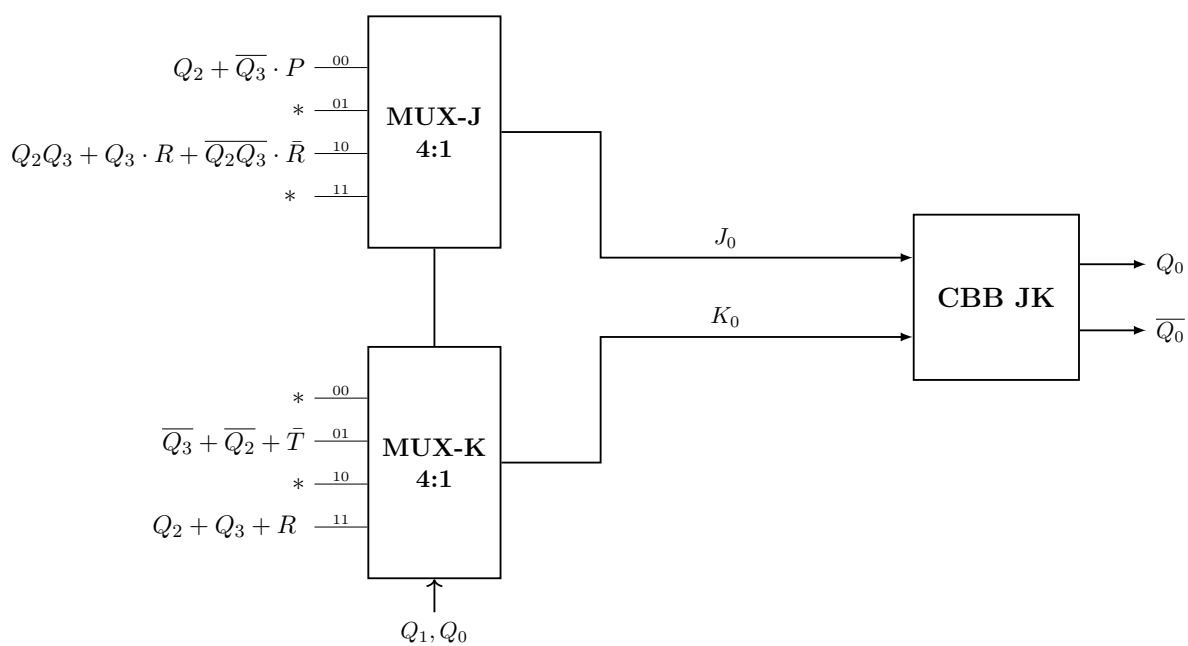


Figura 7: Implementarea grafică a etajului Q_0

6 Implementarea ieșirilor cu porți logice

Deoarece automatul a fost proiectat având ieșiri de tip Moore (asociate stărilor, nu tranzițiilor), funcțiile de ieșire depind exclusiv de starea curentă ($Q_3Q_2Q_1Q_0$).

Conform organigramei și tabelelor de stare, avem următoarele asocieri:

- **ERR_LED** este activ doar în starea **S7 (ERR)** - cod 0111.
- **SPK_EN** și **RX_LED** sunt active doar în starea **S11 (LSTN)** - cod 1011.
- **MIC_EN** și **TX_LED** sunt active doar în starea **S13 (TRSM)** - cod 1101.

Mai jos sunt prezentate hărțile Karnaugh și ecuațiile rezultate pentru fiecare grup de ieșiri.

6.1 Implementarea ieșirii de eroare

Această ieșire semnalizează eșecul împerecherii. Este activă când $Q_3Q_2 = 01$ și $Q_1Q_0 = 11$.

$Q_3Q_2 \backslash Q_1Q_0$	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	1	0	0
10	0	0	0	0

Tabela 6: Harta K pentru ieșirea ERR_LED

Ecuația extrasă:

$$ERR_LED = \overline{Q_3}Q_2Q_1Q_0 \quad (5)$$

6.2 Implementarea ieșirilor de recepție

Aceste semnale activează difuzorul și indicatorul vizual de recepție. Sunt active în starea S11, unde $Q_3Q_2 = 10$ și $Q_1Q_0 = 11$.

$Q_3Q_2 \backslash Q_1Q_0$	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	0	0	1
10	0	0	0	0

Tabela 7: Harta K pentru SPK_EN și RX_LED

Ecuția extrasă:

$$SPK_EN = RX_LED = Q_3 \overline{Q_2} Q_1 Q_0 \quad (6)$$

6.3 Implementarea ieșirilor de transmisie

Aceste semnale activează microfonul și indicatorul vizual de emisie. Sunt active în starea S13, unde $Q_3 Q_2 = 11$ și $Q_1 Q_0 = 01$.

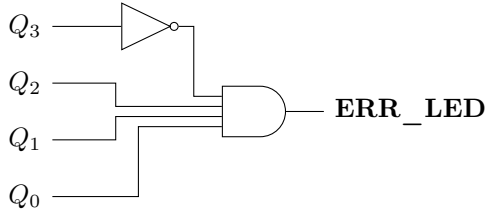
$Q_3 Q_2 \backslash Q_1 Q_0$	00	01	11	10
00	0	0	0	0
01	0	0	1	0
11	0	0	0	0
10	0	0	0	0

Tabela 8: Harta K pentru MIC_EN și TX_LED

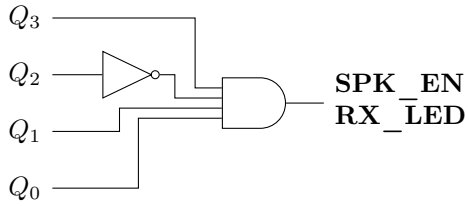
Ecuția extrasă:

$$MIC_EN = TX_LED = Q_3 Q_2 \overline{Q_1} Q_0 \quad (7)$$

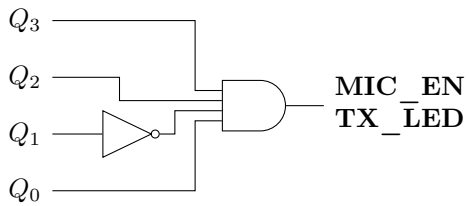
1. Error Output (S7)



2. Reception Output (S11)



3. Transmission Output (S13)



7 Implementarea cu microinstrucțiuni variabile

7.1 Organigrama pentru implementarea cu microinstrucțiuni

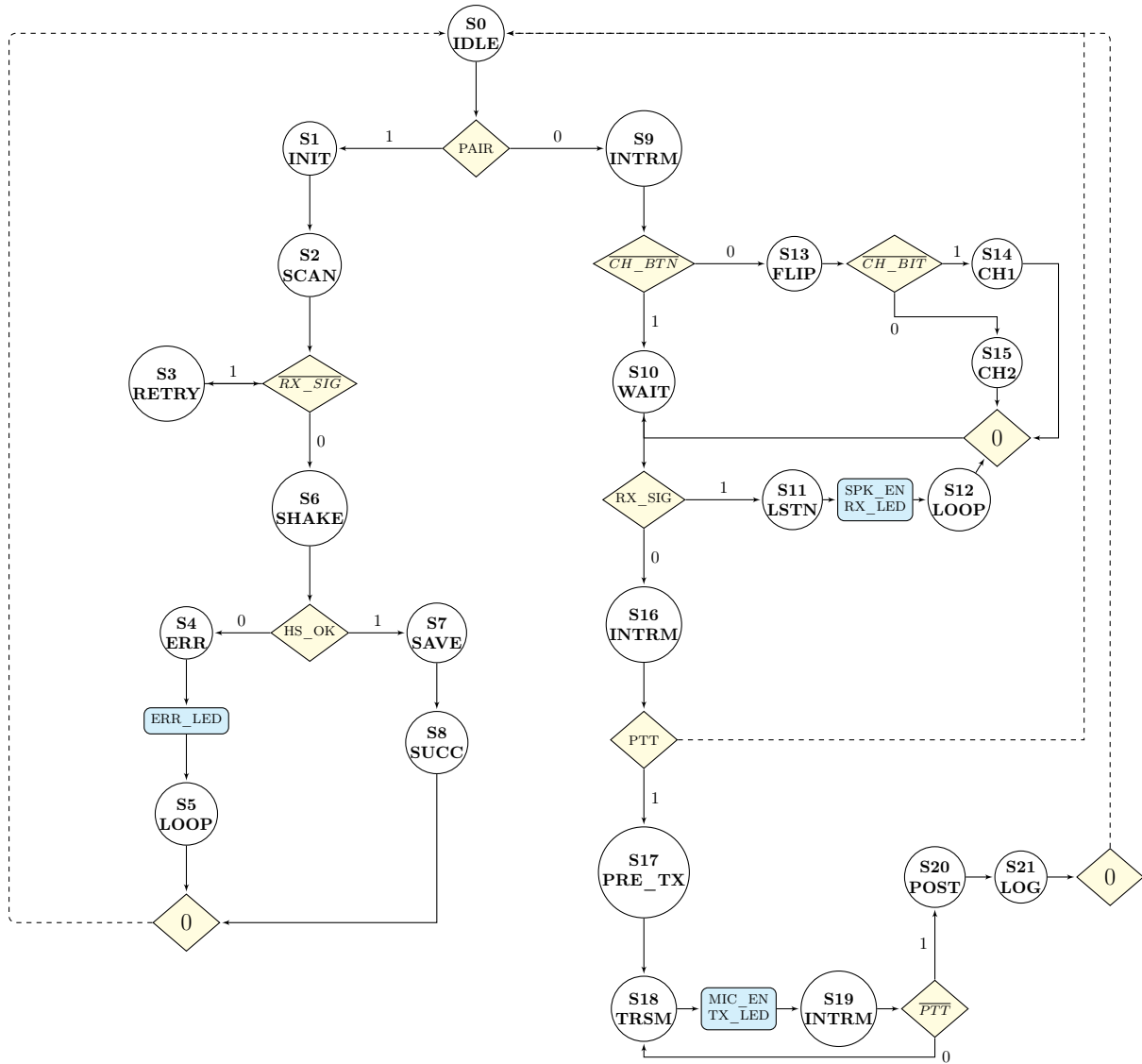


Figura 8: Organigrama adaptată pentru implementarea cu microinstrucțiuni

7.2 Calculul lungimii microinstrucțiunii

7.2.1 Date de intrare

Conform specificațiilor organigramei, parametrii sistemului sunt:

- Număr de stări (N_{stari}): 22
- Număr variabile de intrare (N_{in}): 9
- Număr variabile de ieșire (N_{out}): 5

7.2.2 Dimensionarea câmpurilor

1. **Câmpul de adresare (n_{adr}):** Numărul de biți necesari pentru a adresa unic cele 22 de stări se calculează logaritmice:

$$n_{adr} = \log_2(N_{stari}) = \log_2(22)$$

Deoarece $2^4 = 16 < 22 \leq 32 = 2^5$:

$$n_{adr_necesar} = 5 \text{ biți}$$

Vom alege memoria de tip **64 x 4** pentru implementarea microinstrucțiunilor așa încât:

$$n_{adr} = \log_2(64) = 6 \text{ biți}$$

2. **Câmpul condițiilor de intrare (n_{ci}):** Pentru a selecta una dintre cele 9 variabile de intrare (condiții de test), calculăm:

$$n_{ci} = \log_2(N_{in}) = \log_2(9)$$

Deoarece $2^3 = 8 < 9 \leq 16 = 2^4$:

$$n_{ci} = 4 \text{ biți}$$

Astfel va fi folosit un **MUX 16:1** pentru schema unității de comandă.

3. **Câmpul de ieșire (n_{out}):** În formatul variabil, pentru instrucțiunile de tip 0 (comandă), biții sunt alocați direct variabilelor de ieșire.

$$n_{out} = 5 \text{ biți}$$

7.2.3 Aplicarea calculului final

Lungimea microinstrucțiunii este determinată de maximum dintre lungimea necesară pentru instrucțiunile de salt (Tip 1) și cele de comandă (Tip 0), plus bitul de selecție a formatului:

$$L_{\mu i} = 1 + \max(\underbrace{n_{ci} + n_{adr}}_{\text{Tip 1}}, \underbrace{n_{out}}_{\text{Tip 0}})$$

Înlocuind valorile calculate:

$$L_{\mu i} = 1 + \max(4 + 6, 5)$$

$$L_{\mu i} = 1 + \max(10, 5)$$

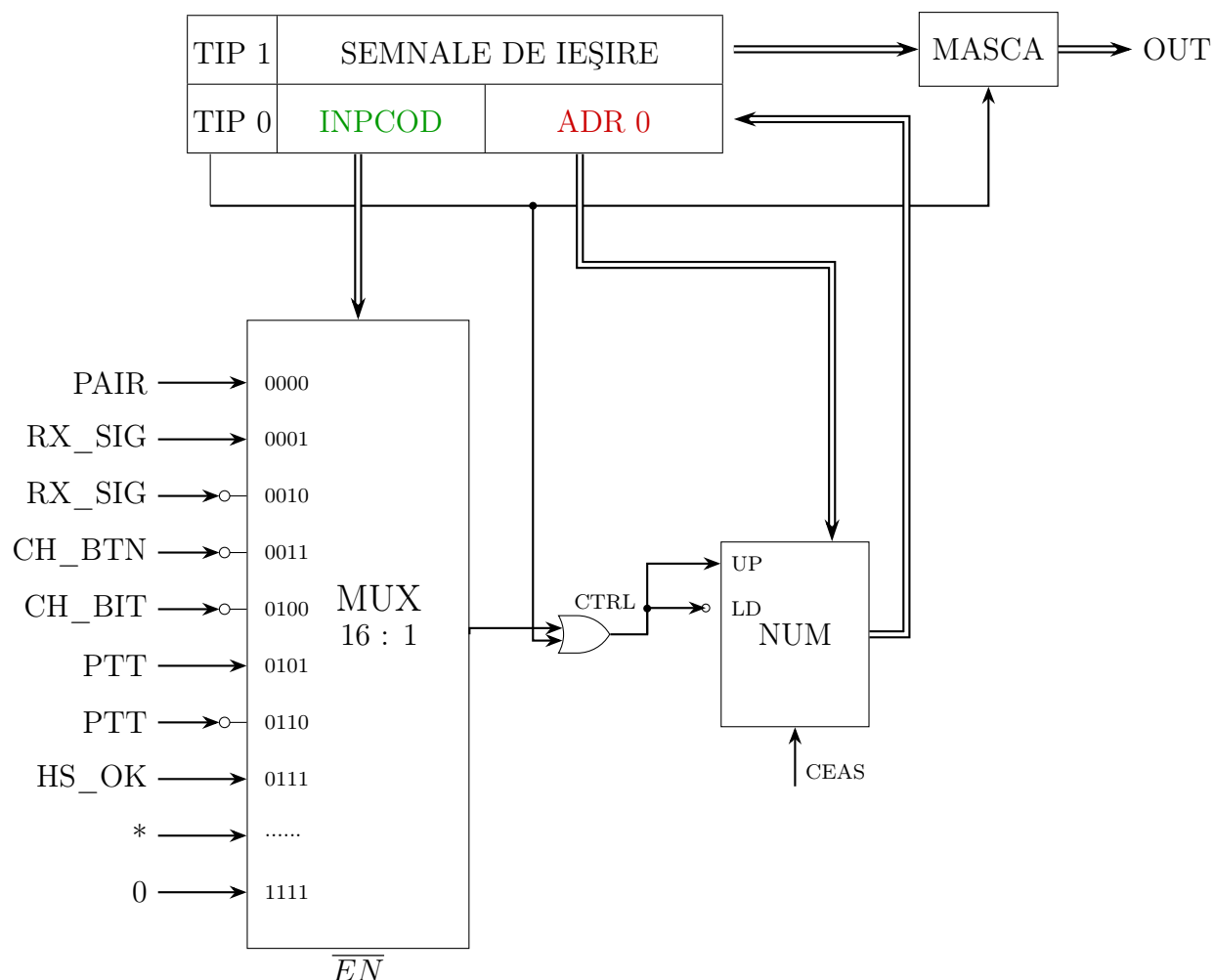
$$L_{\mu i} = 1 + 10 = 11 \text{ biți}$$

Concluzie: Lungimea microinstrucțiunii în format variabil este de **11 biți**, structurată astfel:

- **Bit 10:** Selector format (1 bit).
- **Biții 9-0 (Tip 1):** 4 biți pentru condiționale + 6 biți pentru adresele stărilor (primul bit pentru adresele stărilor va fi mereu 0, deoarece sunt doar 5 biți de adresă necesari).
- **Biții 9-0 (Tip 0):** 5 biți utilizați pentru Ieșiri (restul de 5 biți fiind neutilizați).

Vom concatena **3 circuite** de memorie **64 x 4** pentru implementarea dispozitivului.

7.3 Proiectarea unității de comandă microprogramată



7.4 Alegerea componentelor digitale

7.4.1 Memorie

Pentru implementarea memoriei de comandă, s-au luat în considerare următoarele cerințe:

- **Lungimea microinstrucțiunilor:** 10 biți (necesar ≥ 8).
- **Număr de stări:** 22 (necesar ≥ 16).

Soluția aleasă: S-a optat pentru utilizarea a 3 memorii de 4 biți și 64 cuvinte, model **CAT22C10** (SRAM/EEPROM), conectate în paralel.

- Deși modelul 74HC7403 (Phillips) a fost considerat, CAT22C10 este utilizat mai frecvent.
- Prin legarea în paralel, se obțin disponibili 12 biți, dintre care 2 vor rămâne neutilizați.

7.4.2 Multiplexare

Gestionarea tranzițiilor de stare necesită selectarea sursei pentru adresa următoare.

- **Cerințe:** Sunt necesare 16 intrări (pentru a acomoda cele 9 inputuri de sistem plus logica internă) și 4 biți de selecție.
- **Soluția aleasă:** Un multiplexor 16-la-1 din familia 54150, specific modelul **SN54150**.

7.4.3 Numărătoare

Pentru secvențierea stărilor, este necesar un sistem de numărare capabil să adreseze cele 22 de stări.

- **Adresare:** 22 de stări necesită adrese de 5 biți, deoarece $2^4 = 16 < 22 < 2^5 = 32$.
- **Soluția aleasă:** Două numărătoare de 4 biți conectate în cascadă. S-a decis utilizarea cipului **SN54192** (în detrimentul 74HC161) datorită familiarității.
- **Conexiune:** Pinul de *Carry* al numărătorului de biți mai puțin semnificativi trebuie conectat la pinul *Load/Up* al numărătorului mai semnificativ.

7.4.4 Masca și Porțile Logice

Logica de control și mascare necesită următoarele componente discrete:

- **Porți AND:** Sunt necesare 5 porți (una pentru fiecare output). Se vor folosi 2 cipuri **74HC08** (care oferă un total de 8 porți).
- **Invertor:** Este necesar un invertor pentru intrările multiplexorului, asigurat de un cip **74HC04**.
- **Porți OR:** Este necesar un cip **74HC32** pentru a decide logica dintre incrementarea numărătorului (+1) sau încărcarea adresei de salt (ADR0).

7.4.5 Detalii Tehnice și Pinout

Mai jos sunt detaliile pinilor pentru componentele principale identificate în documentație:

Componentă	Pin	Funcție/Descriere
CAT22C10 (Memorie)	8	V_{SS} (GND)
	18	V_{CC}
	1-7	Adrese ($A_0 - A_5$) și NC
	11	WE (Write Enable)
SN54150 (Multiplexor)	24	V_{CC}
	12	GND
	11, 13-15	Selecție ($S_0 - S_3$)
	10	Output Inverted ($\bar{W}$)
SN54192 (Counter)	16	V_{CC}
	8	GND
	5	UP (Count Up)
	4	DOWN (Count Down)
	11	$\overline{LOAD}$

Tabela 9: Selecție pini relevanți

7.5 Completarea memoriei de microprogram

Stare	ID	TIP 1/0	INPCOD	ADR0	Nume
S_0	00000	0	00000	1001	IDLE
S_1	00001	1	00000	****	INIT
S_2	00010	0	00100	0110	SCAN
S_3	00011	0	00100	0110	RETRY
S_4	00100	1	10000	****	ERROR
S_5	00101	0	11110	0000	LOOP
S_6	00110	0	01110	0100	SHAKE
S_7	00111	1	00000	****	SAVE
S_8	01000	0	11110	0000	SUCCESS
S_9	01001	0	00110	1101	INTER
S_{10}	01010	0	00011	0000	WAIT
S_{11}	01011	1	01010	****	LISTEN
S_{12}	01100	0	11110	1010	LOOP
S_{13}	01101	0	01000	1111	FLIP
S_{14}	01110	0	11110	1010	CH1
S_{15}	01111	0	11110	1010	CH2
S_{16}	10000	0	01010	0000	INTER
S_{17}	10001	1	00000	****	PRE_TX
S_{18}	10010	1	00101	****	TRANSMIT
S_{19}	10011	0	01101	0010	INTER
S_{20}	10100	1	00000	****	POST
S_{21}	10101	0	11110	0000	LOG

Tabela 10: Conținutul memoriei de microprogram